

A Retail Digital Rupiah Prototype on Hyperledger Fabric: Two-Tier Business Processes and a Reproducible Benchmark

1st Muhammad Fachrizal Giffari

Department of Electrical and
Information Engineering
Universitas Gadjah Mada

Yogyakarta, Indonesia

muhammadfachrizalgiffari@mail.ugm.ac.id

2nd Teguh Bharata Adji

Department of Electrical and
Information Engineering
Universitas Gadjah Mada

Yogyakarta, Indonesia

adji@ugm.ac.id

3rd Guntur Dharma Putra

Department of Electrical and
Information Engineering
Universitas Gadjah Mada

Yogyakarta, Indonesia

gdputra@ugm.ac.id

Abstract—Bank Indonesia’s public Project Garuda experiments focus on wholesale central bank digital currency. This paper asks how a testable retail Digital Rupiah prototype can represent a two-tier distribution model and institutional oversight on a permissioned ledger. We implement the prototype with Hyperledger Fabric 2.5, Go chaincode and backend services, CouchDB, and PostgreSQL. The prototype separates institutional and retail roles. Bank Indonesia issues Digital Rupiah into Treasury and distributes it directly to a validator bank or payment service provider acting as custodian. Backend ownership checks and chaincode MSP checks enforce user and organization boundaries, while the shared Fabric channel does not provide transaction privacy. Automated tests evaluate authorization and transfer safety. Hyperledger Caliper evaluates clean-ledger profiles with deterministic seeding, one, two, and four workers, functional workloads, repeated peak rounds, and an aggregate-overspend scenario. The final transfer profiles report 29.1–29.5 TPS at a 30 TPS offered load with an 86.23–100.00% success rate, and 56.3–57.3 TPS at 60 TPS with a 12.05–12.48% success rate. Five peak repetitions average 54.8 TPS completion throughput and 16.33% success. Functional workloads complete 168 of 168 transactions, while boundary and negative-path oracles pass their expected rejections. These measurements describe a single-host prototype and do not establish production capacity or cross-machine reproducibility.

Index Terms—Digital Rupiah, retail CBDC, Hyperledger Fabric, two-tier distribution, performance evaluation

I. INTRODUCTION

Central banks are evaluating Central Bank Digital Currency (CBDC) as a digital form of sovereign money [1]. Bank Indonesia pursues CBDC through Project Garuda. Its publicly documented proof of concept targets the *wholesale* tier, specifically a cash ledger for issuance and redemption among the central bank and large financial institutions [2], [3]. The Garuda White Paper also describes a retail tier in which intermediary banks and payment service providers (PJP) distribute Digital Rupiah to households and merchants [3]. That description remains at the policy and architecture level, and we are not aware of an openly available, testable system that implements Bank Indonesia’s retail money business processes end to end on a blockchain ledger.

Central-bank digital money involves a sequence of business processes, not a single ledger operation. The central bank *issues* money into its Treasury and *distributes* liquidity directly to a validator bank or PJP custodian. Custodians serve customers and merchants in the retail tier. Holders *transact*, custodians *redeem* money to the central bank, and the supervisor *reconciles* and audits its circulation. A blockchain implementation must express these processes as deterministic transactions that each organization can validate independently. It must also preserve two-tier intermediation to reduce bank disintermediation [4] and give the supervisor an audit surface.

Most technical studies isolate one part of this problem: permissioned CBDC architecture [5], privacy [6], security threat models [7], or throughput in account-based systems [8]. Deployed retail systems, including Nigeria’s eNaira [9], China’s e-CNY [10], and the Bahamian Sand Dollar [11], publish design papers but not reproducible performance benchmarks. The surveyed literature therefore provides no integrated, openly measurable retail reference for the Indonesian two-tier model [12], [13].

To address this gap, we implement a Digital Rupiah prototype on a permissioned Hyperledger Fabric network [14]. The prototype maps Bank Indonesia’s business processes to chaincode transactions. This paper contributes:

- 1) A two-tier Digital Rupiah chaincode that integrates hierarchical issuance and distribution, retail transfers, and supervisor-facing reconciliation. This extends the wholesale-only Garuda proof of concept with a testable retail implementation.
- 2) A deterministic transfer protocol with explicit authorization and balance checks, Fabric multi-version concurrency control (MVCC), and atomic wallet updates. The protocol lets endorsing organizations apply the same state-transition rules independently.
- 3) An open, reproducible retail-CBDC benchmark with Hyperledger Caliper [15] that documents the testbed, workload, deterministic population seeding, offered-load profiles, and authorization and aggregate-overspend

checks. The benchmark reports throughput, latency, success rate, and container-level resource measurements while keeping the local testbed and one-worker scope explicit.

II. BACKGROUND AND RELATED WORK

A. Two-Tier Retail CBDC

In the two-tier (intermediated) model the central bank issues CBDC and remains the ultimate liability holder, while regulated intermediaries serve end users and operate customer-facing rails [1], [4]. This preserves the existing division of labor and limits the central bank’s exposure to retail operations. Bank Indonesia adopts this structure for Digital Rupiah, with banks and PJP acting as distributors between the central bank and the public [3].

B. Permissioned Blockchain and Hyperledger Fabric

Hyperledger Fabric is a modular, permissioned blockchain with pluggable ordering, channel-based data isolation, and separate execution, ordering, and validation stages [14]. Membership Service Providers (MSPs) govern identity, the crash-fault-tolerant Raft protocol orders transactions, and CouchDB can provide a queryable world state. Fabric has consequently been used in CBDC and regulated-finance prototypes [5], [16]. Hyperledger Caliper [15], [17] measures throughput and latency under different workloads.

C. Authorization and Supervision

Retail CBDC systems require explicit authorization boundaries, safe balance updates, and supervisory visibility. Token-based retail designs also face denial-of-service and double-spend risks [18], so transfer validation must be deterministic. Our chaincode combines organization-aware authorization, wallet-state checks, and supervision events, complementing studies that address privacy [6] or security [7] separately. The closest published prototype is Project Aurum, a full-stack two-tier retail CBDC built for the Hong Kong Monetary Authority [19], which pairs a wholesale interbank system with a retail e-wallet system and, like this work, keeps the two tiers architecturally distinct. Aurum reports no reproducible throughput benchmark, and its source code and technical manuals are released only to BIS member central banks rather than publicly. What distinguishes this prototype among the representative systems in Table I is that its two-tier intermediation and authorization and supervision controls are realized in a publicly available implementation with a reproducible benchmark, rather than described in design documentation alone.

III. BANK INDONESIA BUSINESS PROCESSES

The system implements six processes in the circulation of central-bank digital money. Fig. 1 shows the actors in each tier. Table II identifies the initiator, chaincode transaction, and endorsement requirement for each process.

TABLE I
COMPARISON WITH REPRESENTATIVE RETAIL CBDC SYSTEMS

System	Platform	2-tier	Open	Bench.
e-CNY [10]	Permissioned	✓	×	×
eNaira [9]	Fabric	✓	×	×
Sand Dollar [11]	Permissioned	✓	×	×
OpenCBDC [8]	Account	×	✓	✓
This work	Fabric 2.5	✓	✓	✓

Open: implementation publicly available. Bench.: open, reproducible throughput/latency benchmark. OpenCBDC is account-based and not distributed-ledger-based. Rows span deployed national CBDCs and a research platform, and summarize public information rather than a controlled comparison.

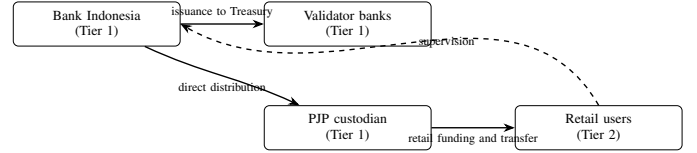


Fig. 1. Two-tier actor structure of the Digital Rupiah prototype. Bank Indonesia issues Digital Rupiah into Treasury and distributes it directly to a validator bank or PJP custodian, which serves retail users. Dashed flow denotes supervisory visibility.

TABLE II
BANK INDONESIA BUSINESS PROCESSES AND THEIR CHAINCODE REALIZATION

Process	Initiator	Chaincode transaction
Issuance	Bank Indonesia	Mint, RequestIssuance
Distribution	Bank Indonesia	DistributeToParticipant
Wallet creation	Bank / PJP	CreateWallet
Payment	Customer	Transfer
Redemption	Participant	Burn, RequestRedemption
Supervision	OJK / BI	GetReconciliationReport, GetMetrics, GetSupervisionEvents

The channel endorsement policy accepts an institutional MSP from Bank Indonesia, HimbaraBank, CommercialBank, or PJP; OJK is observer-only.

The institutional tier contains Bank Indonesia, validator banks, and PJP Custodians. These participants follow a life-cycle of submission, approval, freezing, unfreezing, rejection, and offboarding. Bank Indonesia mints Digital Rupiah only to the Treasury wallet, then distributes liquidity directly to an active validator bank or PJP through `DistributeToParticipant`. Each Custodian serves customers and merchants in the retail tier, where the shared transfer path applies ownership, wallet-status, authorization, and balance checks.

IV. SYSTEM ARCHITECTURE

Fig. 2 shows the four-layer architecture: client, backend API, off-chain store, and Fabric network. The web client calls a Go REST backend built with Gorilla Mux. The backend authenticates requests with JWT and role-based access control, stores account and authentication data in PostgreSQL, and submits ledger transactions through the Fabric Gateway SDK. Fig. 3 details the permissioned network.

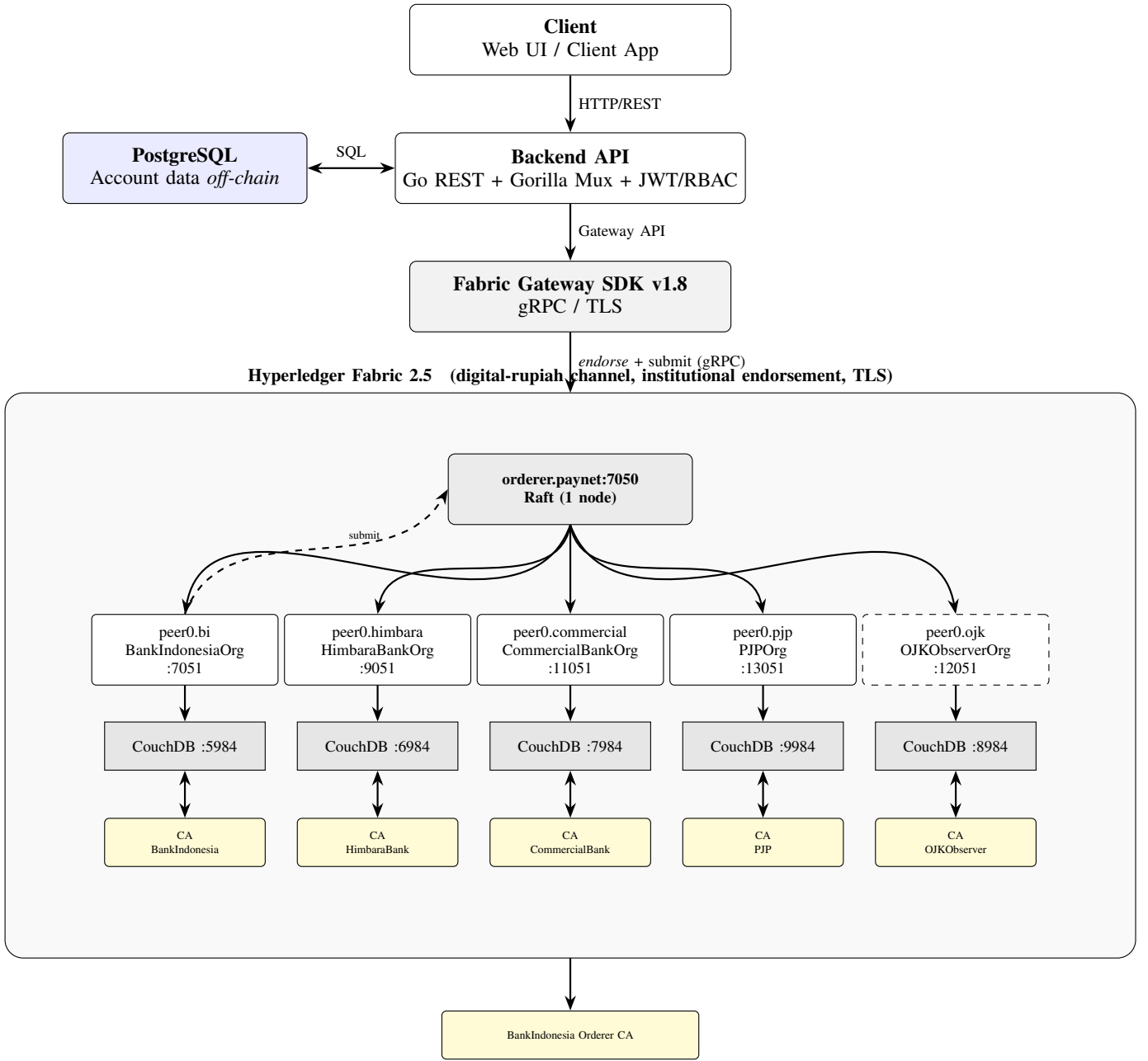


Fig. 2. Full system architecture of the Digital Rupiah prototype: client layer, Go REST backend with JWT/RBAC, off-chain PostgreSQL store, Fabric Gateway SDK, and the Hyperledger Fabric 2.5 network of five peer organizations with CouchDB, a per-organization CA, and a Raft orderer.

A. Network Topology

The network runs five peer organizations on one application channel (Fig. 3, Table III). *BankIndonesiaOrg* is the issuer; *HimbaraBankOrg* (HIMBARA, Indonesia’s association of state-owned banks) and *CommercialBankOrg* are validator and distributor banks; *PJPOrg* represents the payment service provider; and *OJKObserverOrg* is the supervisor (OJK, *Otoritas Jasa Keuangan*, the financial-services authority), which initiates no business transactions and is excluded from chaincode approval. Each organization operates one peer with a CouchDB world state and maintains its own certificate

authority and MSP for identity management. A single Raft orderer orders transactions; this configuration provides ordering but not crash fault tolerance, which requires at least three consenters. The channel endorsement policy accepts one of the four institutional MSPs, while the chaincode lifecycle policy requires Bank Indonesia and one institutional participant.

B. On-Chain and Off-Chain Data

The Go chaincode in `digital_rupiah.go` groups functions for participants, wallets, issuance and redemption, retail transfers, and supervision (Table IV). Account and authentication data remain in the off-chain PostgreSQL store, while

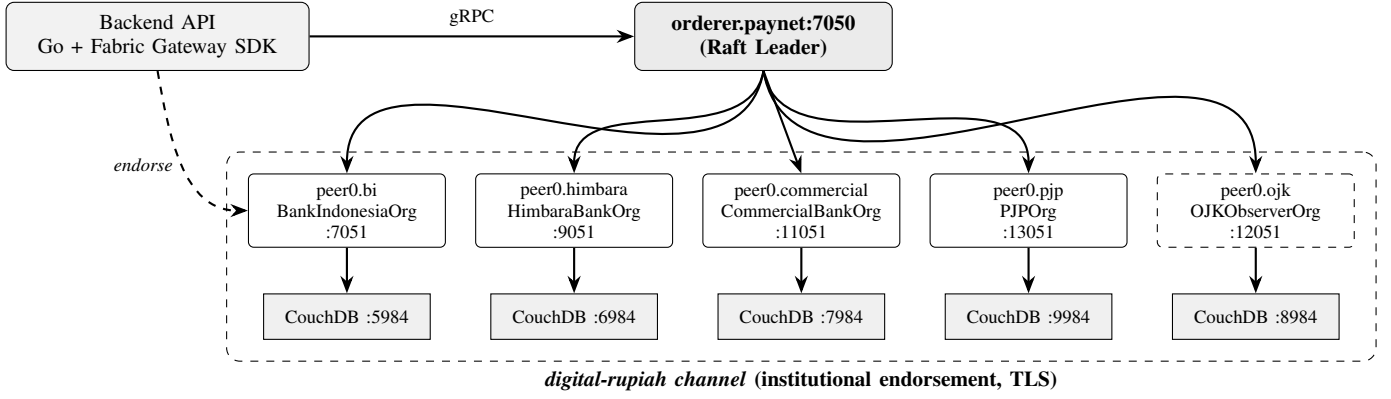


Fig. 3. Hyperledger Fabric network topology: five peer organizations, each with a CouchDB world state, on a single channel under a Raft orderer; the Go backend endorses via the Fabric Gateway SDK.

TABLE III
NETWORK TOPOLOGY OF THE FIVE-ORGANIZATION FABRIC NETWORK

Organization	Role	Peer	CouchDB
BankIndonesiaOrg	Issuer	peer0.bi:7051	:5984
HimbaraBankOrg	Validator	peer0.himbara:9051	:6984
CommercialBankOrg	Validator	peer0.commercial:11051	:7984
PJPOrg	Distributor	peer0.pjp:13051	:9984
OJKObserverOrg	Supervisor	peer0.ojk:12051	:8984

Ordering: orderer.paynet:7050 (single-node Raft); per-org CA; TLS enabled

TABLE IV
CHAINCODE FUNCTION GROUPS

Group	Representative functions
Participants	SubmitParticipant, ApproveParticipant, Freeze/Unfreeze, Offboard
Wallets	CreateWallet, CreateWholesaleWallet, Freeze/UnfreezeWallet
Issuance	Mint, RequestIssuance, RequestIssuanceRtgs
Redemption	Burn, RequestRedemption
Distribution	DistributeToParticipant
Transfer	Transfer
Supervision	GetReconciliationReport, GetMetrics, GetSupervisionEvents

the ledger contains the wallet and transaction state needed for settlement and oversight.

A Go backend enforces role-based access control over six roles, summarized in Table V, before any transaction reaches the chaincode.

The two layers enforce different things, and the split matters for how the prototype's guarantees should be read. The backend maps JWT roles to permitted operations and restricts customers to their own wallets. The chaincode checks the caller's organization, wallet status, transfer amount, and available balance before it writes the state transition. Every endorsing peer re-evaluates these rules independently, while the channel endorsement policy restricts validation to institutional MSPs. The single backend and PostgreSQL instance remain

TABLE V
BACKEND ROLES AND REPRESENTATIVE PERMISSIONS

Role	Representative permitted actions
Public	Health check, login
Authenticated	View own profile, list wallets
Bank/PJP	Create wallet, fund retail wallets
Bank Indonesia	Issue, redeem liquidity, approve participant
Merchant	Initiate transfer, view balance
Supervisor	Reconciliation, metrics, supervision events

trust and availability concentration points for this prototype.

V. PROTOCOL AND DETERMINISM

The transfer path applies explicit authorization and balance checks during endorsement. We formalize these checks and examine their behavior under concurrent transactions.

A. Transfer-Validity Rules

For a transfer of amount a from sender s to receiver r , the chaincode admits the transfer only if every one of the following checks passes:

- the caller's organization and wallet ownership satisfy the authorization rules;
- neither the sender's nor the receiver's wallet is frozen;
- $a > 0$ and the required business reference is present;
- the sender has enough balance to cover a ; and
- the sender and receiver wallet records can be updated atomically.

If any check fails, the transfer is rejected. A successful transfer decreases the sender balance, increases the receiver balance, records the transaction, and emits the supervision event used by the read-only observer path. Treasury distribution accepts only active validator-bank or PJP custodians, and a PJP cannot call issuance directly.

B. Determinism and Double-Spend Safety

Every endorsing peer evaluates the authorization and transfer rules against the same transaction arguments and wallet

```

1: function TRANSFER(sender, receiver, amount)
2: if caller unauthorized or either wallet is frozen then
3:   return REJECT
4: end if
5: if amount  $\leq 0$  or reference is missing then
6:   return REJECT
7: end if
8: if sender.balance < amount then
9:   return REJECT
10: end if
11: sender.balance  $\leftarrow$  sender.balance  $-$  amount
12: receiver.balance  $\leftarrow$  receiver.balance  $+$  amount
13: append transaction history and emit supervision event
14: PutState (validated by Fabric MVCC)
15: return COMMIT

```

Fig. 4. Authorized, atomic, and double-spend-safe transfer executed by the chaincode. Fabric MVCC validates the wallet versions before commit.

records. The state transition does not depend on a peer-local clock or on a peer-local cache. Each successful transfer updates the sender and receiver records in one transaction, so every endorser evaluates the same balance transition before validation.

Double-spend safety follows from Fabric’s multi-version concurrency control (MVCC). An endorsement reads the current version of each wallet record, and Fabric commits the transaction only if those versions remain unchanged through ordering and validation. Two concurrent transfers from the same wallet may endorse against the same record and version, but only the first ordered transaction commits. The second fails MVCC validation after the first update. Because the balance records are the keys in the read set, one version check covers the state used by the transfer and prevents a stale balance update.

VI. PERFORMANCE EVALUATION

A. Methodology

We evaluate the final prototype with Hyperledger Caliper 0.6.0 on a clean ledger for each profile. The combined manifest 20260909-final-evidence-manifest.json records suite seed 20260909 (profile ordering); the transfer configurations retain workload seed 20260725. It also records chaincode digital-rupiah version 3.0 sequence 1, runtime versions, input hashes, result hashes, and the single-host topology. The testbed has five peer organizations, one peer per organization, one Raft orderer, and one CouchDB instance per peer.

The transfer profiles use 70% customer-to-customer transfers, 25% customer-to-merchant transfers, and 5% wallet reads. Each profile warms the network for 30 seconds at 10 TPS, then measures 120-second rounds at 30 and 60 TPS. The one-, two-, and four-worker profiles use the populations and transaction slots specified in their individual configuration files. Sender wallets receive a larger initial balance than receiver wallets, so valid traffic does not begin with a receiver

TABLE VI
FINAL TESTBED CONFIGURATION

Component	Specification
Processor	Intel Core i7-7700HQ, 4 cores, 8 threads, 2.80 GHz
Memory	15.52 GiB
Runtime	Node.js 22.23.2, npm 10.9.1, Docker 27.5.1
Fabric / CouchDB	2.5 / 3.2
Caliper	0.6.0
Chaincode	digital-rupiah v3.0, sequence 1
Ordering	Single-node Raft; batch timeout 2 s; maximum 10 messages
Endorsement	One institutional MSP: BI, HimbaraBank, CommercialBank, or PJP

TABLE VII
FINAL TRANSFER-PROFILE RESULTS

Workers	Load	Succ.	Fail	Success (%)	Send (TPS)	Throughput (TPS)	Avg. lat. (s)
1	30 TPS	3,105	496	86.23	30.0	29.1	0.71
1	60 TPS	898	6,300	12.48	60.0	56.3	6.36
2	30 TPS	3,602	0	100.00	30.0	29.5	0.40
2	60 TPS	874	6,327	12.14	60.0	57.2	6.48
4	30 TPS	3,486	118	96.73	30.0	29.5	0.76
4	60 TPS	868	6,336	12.05	60.0	57.3	6.47

TABLE VIII
FAILURE DIAGNOSTICS IN FINAL TRANSFER PROFILES

Profile	Status 11	Gateway limit	Total
1 worker	5,332	1,464	6,796
2 workers	4,919	1,408	6,327
4 workers	5,121	1,333	6,454
Five repetitions	11,878	3,191	15,069

at its maximum balance. Caliper records successful and failed submissions, completion throughput, and latency.

All warmup rounds completed without failure. At 30 TPS, completion throughput was 29.1–29.5 TPS and success rate was 86.23–100.00%. At 60 TPS, completion throughput was 56.3–57.3 TPS while success rate was 12.05–12.48%. Completion throughput includes failed submissions, so it is not successful throughput.

The final transfer logs classify every failed submission as Fabric status 11 or the gateway message too many requests for /gateway.Gateway. Status 11 is the Fabric validation code for an MVCC read conflict: a transaction read a wallet version that changed before commit. The gateway message identifies client-path concurrency pressure. The logs contain no insufficient-balance or transport failure in these final transfer profiles.

Five repetitions on one clean ledger produced a mean completion throughput of 54.8 TPS and a mean success rate of 16.33%. The success-rate range was 11.19–24.85%, and mean latency ranged from 3.23 to 6.93 s. These values describe variation in the reported local configuration.

The four functional profiles completed all 168 transactions without failure. They verify onboarding, monetary operations, read-only supervision, and policy updates at their configured rates. Their throughputs are not combined with the transfer

TABLE IX
FIVE REPETITIONS AT AN OFFERED LOAD OF 60 TPS

Round	Succ.	Fail	Success (%)	Throughput (TPS)	Avg. lat. (s)
1	895	2,707	24.85	57.7	3.23
2	403	3,199	11.19	54.9	6.93
3	598	3,004	16.60	54.8	4.93
4	448	3,154	12.44	52.7	6.39
5	597	3,005	16.57	54.1	4.50
Mean	588.2	3,013.8	16.33	54.8	5.20

TABLE X
FUNCTIONAL CALIPER PROFILES

Profile	Succ.	Fail	Throughput (TPS)	Avg. lat. (s)
KYC onboarding	48	0	4.3	0.74
Monetary operations	40	0	19.4	0.33
Supervision reads	40	0	30.6	0.03
Administrative policy	40	0	5.1	1.05

TABLE XI
SEMANTIC CONFORMANCE PROFILES

Profile	Succ.	Fail	Oracle result
Authorization	2	1	PASS; one expected custodian-mismatch rejection.
Boundary	4	4	PASS; 8/8 cases, zero mismatches, zero infrastructure errors.
Negative path	0	7	PASS; 7/7 expected rejection rules enforced.
Aggregate overspend	10	190	PASS; final sender balance Rp40,000.

TABLE XII
MAXIMUM MONITORED RESOURCE USAGE

Profile	CPU max. (%)	Memory max. (MB)
Transfer, 1 worker	2.35	163
Transfer, 2 workers	2.74	166
Transfer, 4 workers	2.69	163
Five peak repetitions	2.53	170

sweep because the functions and read-write patterns differ.

The failed counts in these semantic profiles represent expected policy rejections. The aggregate-overspend workload requested Rp200,000 from an initial Rp50,000 sender balance. Ten transfers committed and 190 were rejected; the final balance remained Rp40,000. These observations support the tested authorization and balance invariants, but do not constitute a formal security proof.

No monitored container approached the host CPU or memory capacity. The observed drop in success at 60 TPS therefore cannot be attributed to simple resource exhaustion alone; the logs show state conflicts and gateway concurrency pressure. The evidence supports the architecture and policy controls, then bounds their observed behavior under the tested local loads.

VII. LIMITATIONS AND FUTURE WORK

The evaluation uses a single host with one peer per organization and no geographic distribution or network latency. Its Caliper results therefore apply only to the reported configuration and do not estimate national-scale performance. Each

transfer profile uses a clean ledger and a fixed seeded population, while the final sweep varies one, two, and four workers. A distributed deployment and a graded-skew contention workload would be needed to characterize other operating conditions.

The shared Fabric channel does not provide transaction privacy, and the prototype does not use private data collections or privacy-preserving cryptography. The single orderer provides ordering but does not tolerate its own failure. The prototype also omits offline payment and leaves connections to domestic infrastructure such as BI-RTGS and BI-FAST as stubs.

The authorization and aggregate-overspend profiles cover the tested paths only. They do not establish formal security, production readiness, or cross-machine reproducibility. Future work should repeat the protocol on distributed hosts, instrument endorsement and commit phases, expand the contention workload, and evaluate privacy, offline payment, and interoperability.

VIII. CONCLUSION

This paper implements Bank Indonesia’s digital money business processes in a two-tier Digital Rupiah prototype on a five-organization Hyperledger Fabric network. The chaincode integrates hierarchical issuance and distribution, retail transfers, organization-aware authorization, and supervisor-facing reconciliation.

The transfer protocol combines explicit wallet-state checks with Fabric MVCC. The final Caliper sweep reports 29.1–29.5 TPS completion throughput at a 30 TPS offered load with an 86.23–100.00% success rate. At 60 TPS, completion throughput is 56.3–57.3 TPS and success rate is 12.05–12.48%. Five peak repetitions average 54.8 TPS completion throughput and 16.33% success. The functional profiles complete 168 of 168 transactions, and the authorization, boundary, negative-path, and aggregate-overspend oracles pass their expected checks. These results provide a local, auditable baseline for retail Digital Rupiah experiments, not a production-capacity claim.

ARTIFACT AVAILABILITY

The chaincode, the five-organization network definition, the Go backend, and the Hyperledger Caliper workloads and benchmark configurations used in this paper are available at <https://github.com/mfachrizalg/bi-coin>.

REFERENCES

- [1] J. Kiff, J. Alwazir, S. Davidovic, A. Farias, A. Khan, T. Khiaonarong, M. Malaika, H. Monroe, N. Sugimoto, H. Tourpe, and P. X. Zhou, “A survey of research on retail central bank digital currency,” *IMF Working Papers*, 2020.
- [2] Bank Indonesia, “Proyek garuda: Wholesale digital rupiah cash ledger — proof of concept report,” Bank Indonesia, Jakarta, Tech. Rep., 2024.
- [3] —, “Proyek garuda: Menavigasi arsitektur digital rupiah — white paper central bank digital currency,” Bank Indonesia, Jakarta, Tech. Rep., 2022.
- [4] R. A. Auer and R. Böhme, “The technology of retail central bank digital currency,” *BIS Quarterly Review*, pp. 85–100, Mar. 2020. [Online]. Available: https://www.bis.org/publ/qtrpdf/r_qt2003j.htm

- [5] Bhawana and S. Kumar, "Permission blockchain network based central bank digital currency," in *2021 IEEE 4th International Conference on Computing, Power and Communication Technologies (GUCON)*. IEEE, 2021, pp. 1–6.
- [6] M. M. Islam and H. P. In, "A privacy-preserving transparent central bank digital currency system based on consortium blockchain and unspent transaction outputs," *IEEE Transactions on Services Computing*, vol. 16, no. 4, pp. 2372–2386, 2023.
- [7] J. Hans, S. Khan, and D. Svetinovic, "Blockchain cbdc security threats using stride," in *2023 Fifth International Conference on Blockchain Computing and Applications (BCCA)*. IEEE, 2023, pp. 522–529.
- [8] J. Lovejoy, C. Fields, M. Virza, T. Frederick, D. Urness, K. Karwaski, A. Brownworth, and N. Narula, "A high performance payment processing system designed for central bank digital currencies," *IACR Cryptology ePrint Archive*, vol. 2022, p. 163, 2022. [Online]. Available: <https://eprint.iacr.org/2022/163>
- [9] J. Ree, "Nigeria's enaira, one year after," International Monetary Fund, IMF Working Paper WP/23/104, 2023.
- [10] People's Bank of China, "Progress of research and development of e-cny in china," People's Bank of China, Tech. Rep., 2021. [Online]. Available: <https://www.pbc.gov.cn/en/3688110/3688172/4157443/4293696/2021072014364791207.pdf>
- [11] Central Bank of The Bahamas, "Project sand dollar: A digital fiat currency for the bahamas," Central Bank of The Bahamas, Nassau, Tech. Rep., 2020. [Online]. Available: <https://www.centralbankbahamas.com/viewPDF/documents/2019-12-25-02-18-11-Project-Sanddollar.pdf>
- [12] V. Sethaput and S. Innet, "Sok: Blockchain applications in central bank digital currencies (cbdc)," in *2025 7th International Conference on Blockchain Computing and Applications (BCCA)*. IEEE, 2025, pp. 195–202.
- [13] J. N. Aryani, "Designing central bank digital currency in indonesia: A critical literature review," 2023, tugas Akhir Sarjana, Sekolah Bisnis dan Manajemen, Institut Teknologi Bandung.
- [14] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich *et al.*, "Hyperledger fabric: A distributed operating system for permissioned blockchains," in *Proceedings of the Thirteenth EuroSys Conference (EuroSys '18)*. Porto, Portugal: ACM, 2018, pp. 1–15.
- [15] Hyperledger Foundation, "Hyperledger caliper: A blockchain performance benchmark framework," 2024, accessed: Jun. 27, 2026. [Online]. Available: <https://hyperledger.github.io/caliper/>
- [16] L. I. Hanif, "Rancang bangun sistem blockchain aplikasi meetcoin menggunakan kerangka kerja hyperledger fabric," Universitas Gadjah Mada, Yogyakarta, Skripsi Sarjana, Departemen Teknik Elektro dan Teknologi Informasi, 2024.
- [17] Hyperledger Performance and Scale Working Group, "Hyperledger blockchain performance metrics," Hyperledger Foundation, Tech. Rep., 2018.
- [18] A. Ahuja, S. Pani, S. Kanchanapalli, V. R. R. M. A, and S. Lodha, "Limbocoin: On the denial-of-service of token based retail cbdc," in *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 2024.
- [19] Bank for International Settlements, "Project aurum: A prototype for two-tier central bank digital currency (cbdc)," BIS Innovation Hub Hong Kong Centre, Hong Kong Monetary Authority and Hong Kong Applied Science and Technology Research Institute, Tech. Rep., Oct. 2022. [Online]. Available: <https://www.bis.org/publ/othp57.pdf>